



| Interfaces

Interfaces: conceito e motivação

- Interfaces são estruturas que definem contratos de comportamento em Java.
- Elas permitem que classes não relacionadas por herança compartilhem funcionalidades comuns.
- Todos os métodos em uma interface são, por padrão, `public` e `abstract`.
- Interfaces promovem:
 - Modularidade e reutilização
 - Independência entre partes do sistema
 - Flexibilidade para substituição de implementações

Por que usar interfaces?

- **Separação de responsabilidades:**
 - Define o "**o que**" sem impor o "**como**"
- **Reutilização de código:**
 - Sem herança direta entre classes
- **Polimorfismo por contrato:**
 - Diferentes implementações, mesma interface
- **Baixo acoplamento:**
 - Facilita manutenção e evolução do código

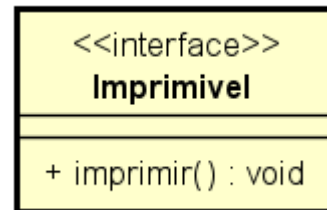
Interfaces vs. Classes Abstratas

Aspecto	Interface	Classe Abstrata
Herança	Pode ser implementada por várias classes	Só pode ser estendida por uma subclasse
Métodos	Apenas assinaturas (exceto <code>default</code> , <code>static</code>)	Pode ter métodos com ou sem implementação
Variáveis	Apenas constantes (<code>public static final</code>)	Pode ter atributos de instância normais
Propósito	Definir contratos	Definir comportamento base comum

Declaração de Interfaces em Java

- Uma interface é declarada com a palavra-chave `interface`.
- Por convenção, seu nome costuma ser um comportamento: `Imprimivel`, `Conectavel`, `Atualizavel`.
- Interface contém **apenas métodos abstratos e constantes**.
 - A partir do Java 8, pode conter métodos `default` e `static`.

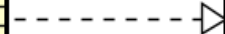
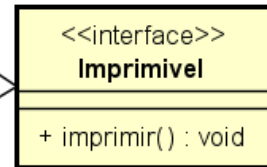
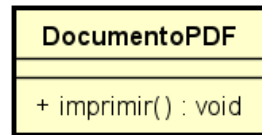
```
public interface Imprimivel {  
    void imprimir();  
}
```



Implementando uma Interface

- Uma classe usa a palavra-chave `implements` para implementar uma interface.
- É obrigatório fornecer uma implementação concreta para todos os métodos da interface.
- A anotação `@Override` é opcional, mas **fortemente recomendada**.

```
public class DocumentoPDF implements Imprimivel {  
    @Override  
    public void imprimir() {  
        System.out.println("Imprimindo documento PDF...");  
    }  
}
```



Regras de Implementação

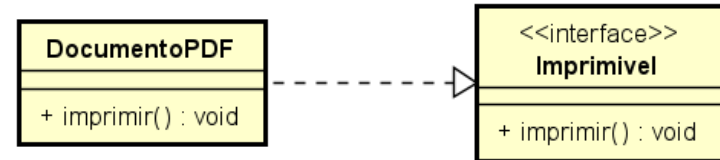
- Todos os métodos da interface:
 - Devem ser implementados na classe concreta.
 - São automaticamente `public` e `abstract`.
- Interfaces não possuem atributos de instância, apenas constantes.
- Uma classe pode implementar várias interfaces, separadas por vírgula.

Uma constante em Java
é declarada com:
`public static final`

```
public class MinhaClasse implements InterfaceA, InterfaceB {  
    // implementação obrigatória de todos os métodos das interfaces  
}
```

Interface como Tipo de Referência

- Interfaces podem ser utilizadas como tipo de referência.
- Isso permite que o mesmo código funcione com diferentes implementações.
- É uma forma de polimorfismo por contrato.



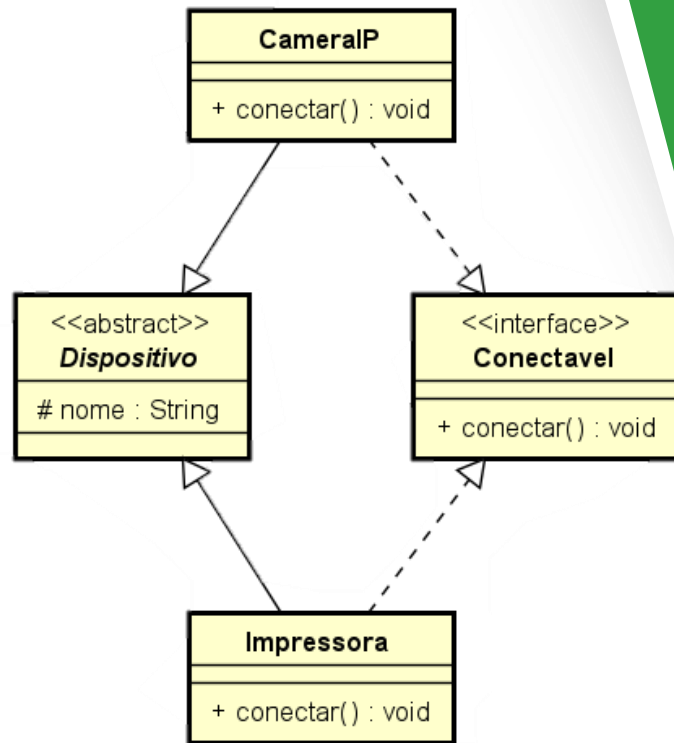
```

Imprimivel arquivo = new DocumentoPDF();
arquivo.imprimir(); // Saída: Imprimindo documento PDF...
  
```

- A chamada `imprimir()` se comporta de forma diferente dependendo do objeto.

Exemplo prático 1: interface Conectavel

- Deseja-se que diferentes dispositivos possam se conectar à rede.
- A interface `Conectavel` garante que qualquer dispositivo que deseje "participar da rede" implemente o método `conectar()`.
- Cada classe oferece sua própria implementação de acordo com o seu contexto.



Exemplo prático 1: interface Conectavel

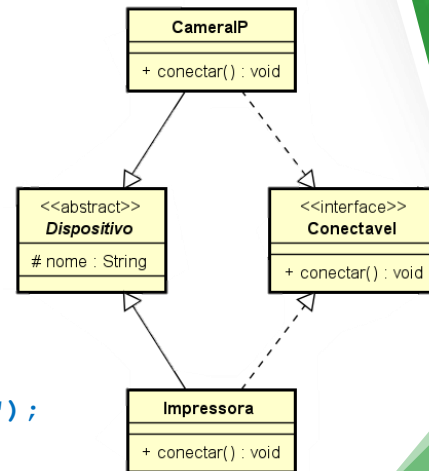
```
// Interface que define o contrato
public interface Conectavel {
    void conectar();
}
```

```
// Classe base
public abstract class Dispositivo {
    protected String nome;
    public Dispositivo(String nome) {
        this.nome = nome;
    }
}
```

// Implementações

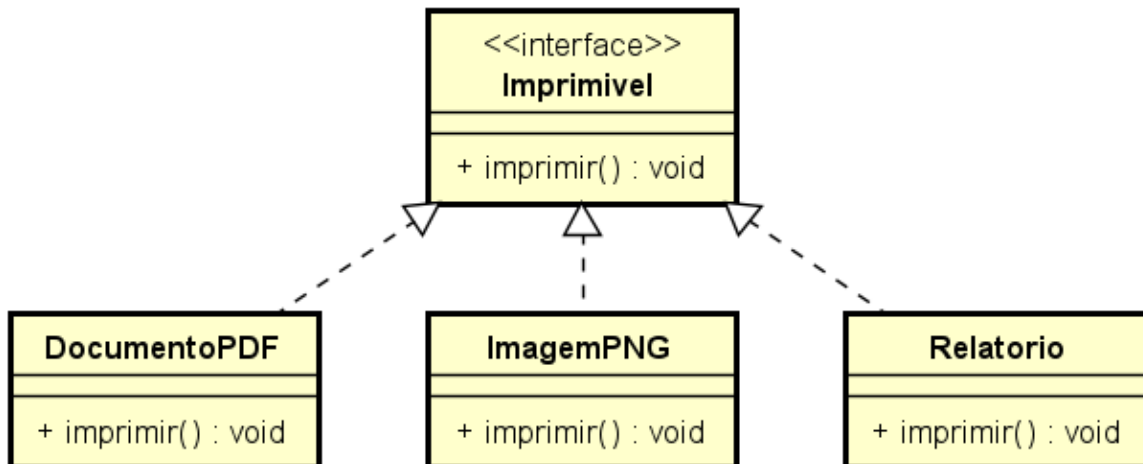
```
public class CameraIP extends Dispositivo implements Conectavel {
    public CameraIP(String nome) { super(nome); }
    public void conectar() {
        System.out.println("Câmera " + nome + " conectada via Wi-Fi.");
    }
}

public class Impressora extends Dispositivo implements Conectavel {
    public Impressora(String nome) { super(nome); }
    public void conectar() {
        System.out.println("Impressora " + nome + " conectada via cabo USB.");
    }
}
```



Exemplo prático 2: interface `Imprimivel`

- A interface `Imprimivel` é usada como tipo genérico.
- Permite adicionar novas classes sem modificar a lógica da aplicação.



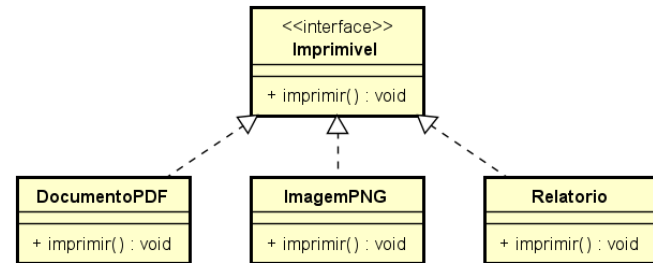
Exemplo prático 2: interface Imprimivel

```
public class DocumentoPDF implements Imprimivel {
    @Override
    public void imprimir() {
        System.out.println("Imprimindo documento PDF...");
    }
}
```

```
public class ImagemPNG implements Imprimivel {
    @Override
    public void imprimir() {
        System.out.println("Imprimindo imagem PNG...");
    }
}
```

```
public class Relatorio implements Imprimivel {
    @Override
    public void imprimir() {
        System.out.println("Imprimindo relatório...");
    }
}
```

```
//Definição da interface
public interface Imprimivel {
    void imprimir();
}
```



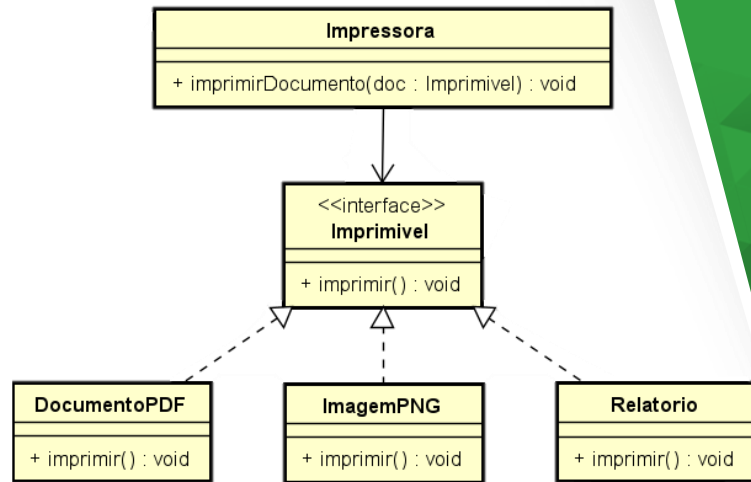
Exemplo prático 2: interface Imprimivel

```
public class Impressora {
    public void imprimirDocumento(Imprimivel doc) {
        doc.imprimir(); // chamada polimórfica
    }

    public static void main(String[] args) {
        Impressora impressora = new Impressora();

        Imprimivel pdf = new DocumentoPDF();
        Imprimivel imagem = new ImagemPNG();
        Imprimivel relatorio = new Relatorio();

        impressora.imprimirDocumento(pdf);
        impressora.imprimirDocumento(imagem);
        impressora.imprimirDocumento(relatorio);
    }
}
```



Polimorfismo com Interface como Tipo de Referência

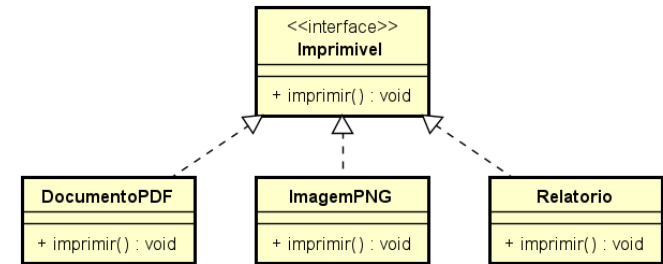
- Interfaces podem ser usadas como tipo genérico em coleções.
- É possível criar uma lista de `Imprimivel` com objetos de classes diferentes.
- Cada item da lista se comportará de forma distinta ao chamar `imprimir()`.

```
import java.util.ArrayList;
import java.util.List;

public class SistemaImpressao {
    public static void main(String[] args) {
        List<Imprimivel> filaDeImpressao = new ArrayList<>();

        filaDeImpressao.add(new DocumentoPDF());
        filaDeImpressao.add(new ImagemPNG());
        filaDeImpressao.add(new SmartImpressora("HP LaserJet M15w"));

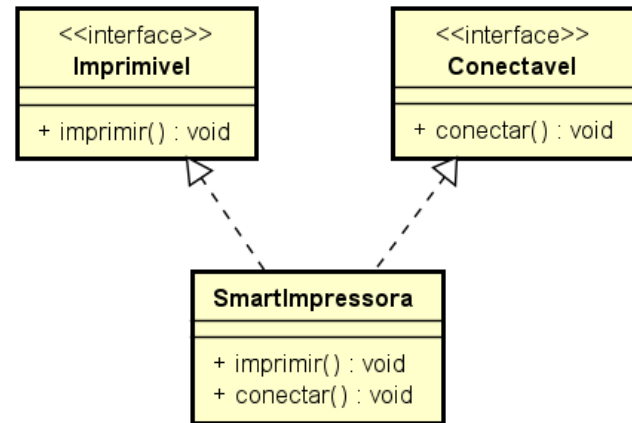
        for (Imprimivel item : filaDeImpressao) {
            item.imprimir(); // comportamento polimórfico
        }
    }
}
```



Qualquer classe que implemente `Imprimivel` pode ser integrada à fila de impressão sem modificar o funcionamento do sistema.

Múltiplas Interfaces em uma Mesma Classe

- Em Java, uma classe pode implementar várias interfaces.
- Isso permite combinar comportamentos distintos de forma modular.
- Exemplo:
 - `Imprimivel`: define comportamento de impressão
 - `Conectavel`: define comportamento de conexão à rede
- A classe `SmartImpressora` combina esses dois contratos em um único objeto funcional.



Múltiplas Interfaces em uma Mesma Classe

```

public interface Imprimivel {
    void imprimir();
}

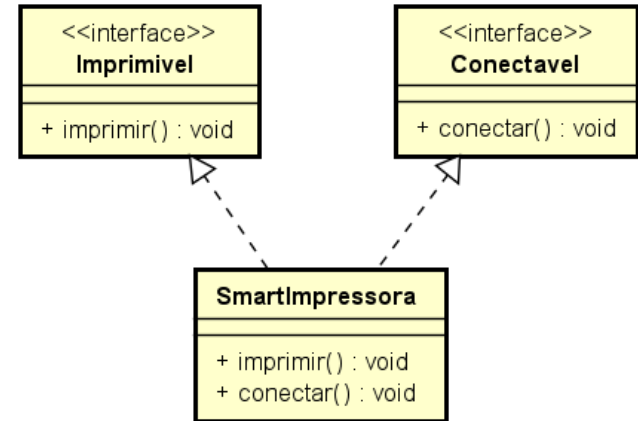
public interface Conectavel {
    void conectar();
}

public class SmartImpressora implements Imprimivel, Conectavel {
    private String modelo;

    public SmartImpressora(String modelo) {
        this.modelo = modelo;
    }

    @Override
    public void conectar() {
        System.out.println("Conectando a impressora " + modelo + " à rede Wi-Fi...");
    }

    @Override
    public void imprimir() {
        System.out.println("Imprimindo documento na impressora " + modelo + "...");
    }
}
  
```



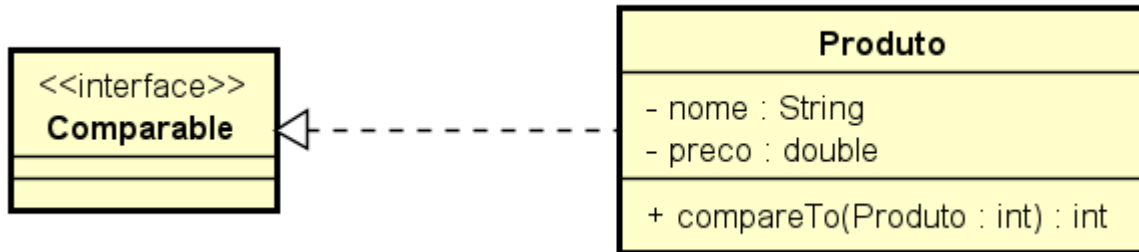
Interfaces da API Padrão do Java

- A linguagem Java fornece várias interfaces prontas, que estruturam funcionalidades comuns de forma padronizada.
- Essas interfaces fazem parte da **Java Standard Library (API)** e são usadas diariamente por desenvolvedores.
- Principais interfaces:
 - `Comparable<T>` – permite ordenar objetos por meio do método `compareTo()`.
 - `Runnable` – define código a ser executado por uma `Thread`.
 - `Serializable` – marca objetos que podem ser convertidos em bytes.
 - `Iterable<T>` – permite iteração com `for-each`.

Ao implementar essas interfaces, sua classe ganha integração direta com estruturas e utilitários da linguagem Java.

API Java - Exemplo: Interface Comparable

- Vamos criar uma classe `Produto` para representar itens de uma loja.
- Queremos ordenar os produtos com base em seu preço.
- Para isso, implementamos a interface `Comparable<Produto>` e sobrescrevemos o método `compareTo()`.



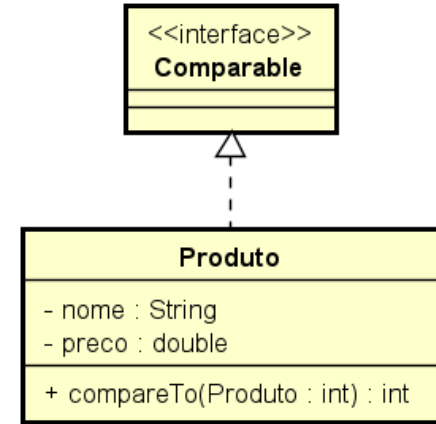
API Java - Exemplo: Interface Comparable

```
public class Produto implements Comparable<Produto> {
    private String nome;
    private double preco;

    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
    }

    @Override
    public int compareTo(Produto outro) {
        return Double.compare(this.preco, outro.preco);
    }

    @Override
    public String toString() {
        return nome + " - R$ " + preco;
    }
}
```

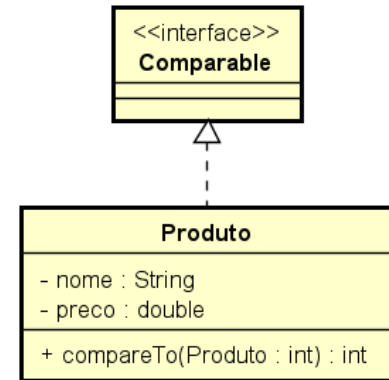


- Método estático (*static*) da classe `Double` do Java
- Retorna:
 - 0, se o primeiro valor é numericamente igual ao segundo
 - **Um valor negativo** se o primeiro é menor que o segundo
 - **Um valor positivo** se o primeiro é maior que o segundo

API Java - Exemplo: Interface Comparable

- O método `Collections.sort()` pode ser usado diretamente em listas de objetos que implementam `Comparable`.
- A ordenação será definida com base na lógica implementada em `compareTo()`.

```
public static void main(String[] args) {  
    List<Produto> produtos = new ArrayList<>();  
    produtos.add(new Produto("Teclado", 120.0));  
    produtos.add(new Produto("Monitor", 899.0));  
    produtos.add(new Produto("Mouse", 59.9));  
  
    Collections.sort(produtos); // ordena pelo preço (compareTo)  
  
    for (Produto p : produtos) {  
        System.out.println(p);  
    }  
}
```



Novidades em Interface a partir do Java 8

- Métodos `default` em Interfaces
 - Introduzidos no Java 8 para permitir métodos com implementação dentro de interfaces.
 - Evitam que todas as classes precisem redefinir comportamentos simples.
 - São úteis para evoluir interfaces sem quebrar código legado.
 - Podem ser sobrescritos pela classe implementadora, se desejado.

Novidades em Interface a partir do Java 8

- Métodos default em Interfaces: Exemplo

```
public interface Atualizavel {  
    default void verificarAtualizacoes() {  
        System.out.println("Verificando atualizações padrão...");  
    }  
}  
  
public class Aplicativo implements Atualizavel {  
    // Usa o comportamento padrão  
}  
  
public class SistemaOperacional implements Atualizavel {  
    @Override  
    public void verificarAtualizacoes() {  
        System.out.println("Buscando atualizações do sistema operacional...");  
    }  
}
```

Novidades em Interface a partir do Java 8

- Métodos `static` em interfaces
 - São usados como métodos utilitários, semelhantes aos de classes helper.
 - Não podem ser sobrescritos.
 - São chamados diretamente via o nome da interface.

Novidades em Interface a partir do Java 8

- Métodos `static` em interfaces: Exemplo

```
public interface ConversorUnidades {  
    static double celsiusParaFahrenheit(double celsius) {  
        return (celsius * 9/5) + 32;  
    }  
}  
  
public class TesteConversao {  
    public static void main(String[] args) {  
        double f = ConversorUnidades.celsiusParaFahrenheit(25);  
        System.out.println("25°C = " + f + "°F");  
    }  
}
```


Novidades em Interface a partir do Java 8

- Interfaces vs. Classes Abstratas com métodos `default` e `static`

Critério	Interface (com <code>default</code> e <code>static</code>)	Classe Abstrata
Herança múltipla	✓ Uma classe pode implementar várias interfaces	✗ Uma classe só pode estender uma única superclasse
Palavra-chave de uso	<code>interface</code>	<code>abstract class</code>
Métodos com corpo	✓ Pode ter <code>default</code> e <code>static</code> , mas não pode ter métodos com estado	✓ Pode ter métodos concretos com acesso ao estado da classe
Atributos/Estado	✗ Não pode ter atributos de instância (somente constantes <code>public static final</code>)	✓ Pode ter atributos de instância e manipulá-los
Construtores	✗ Não possui construtor	✓ Pode ter construtor e inicializar estado
Encapsulamento	✗ Todos os métodos são implicitamente <code>public</code>	✓ Pode usar modificadores (<code>private</code> , <code>protected</code> , etc.)
Objetivo principal	Definir contratos de comportamento	Definir uma base de implementação compartilhada
Abstração de comportamento	✓ Foca no <u>o que</u> fazer	✓ Foca no <u>o que</u> e <u>como</u> fazer (parcialmente)

Exercícios propostos

Verificar atividade no Moodle